



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Día, Fecha:	18 de Agosto de 2026
Hora de inicio:	9:00 - 10:40

Lenguajes Formales y De Programación B-

José Estuardo Orellana Leonardo

Iniciamos a las 09:05

Procesadores de Lenguaje



Lenguajes Formales y de Programación

Escuela de Ingeniería de Ciencias Y Sistemas

Facultad de Ingeniería

Universidad de San Carlos de Guatemala

Agenda



RECORDATORIOS

Foro 5



Intérprete



Compilador



Estructura de un compilador



Diferencias y Ejemplos



Herramientas



Jerarquía de Chomsky y Clasificación de Gramáticas



HORARIOS Y HOJA
DE CALIFICACIÓN

Recursos

Link del Video



<https://www.youtube.com/watch?v=dlgKBNuagIE>

<https://www.youtube.com/watch?v=hzJ8CNdPElc>

Dudas clase anterior ?

Tema visto: Lenguajes natural, formal y de programación,
paradigmas y generaciones



COMPETENCIAS QUE DESARROLLAREMOS

Comprender el funcionamiento del lenguaje de programación aplicándolo a problemas de las fases del compilador.



INTÉRPRETE

Un intérprete es un procesador de lenguaje que traduce y ejecuta el código fuente instrucción por instrucción, sin generar un archivo ejecutable independiente.

Traduce y ejecuta en tiempo real, línea por línea.

Detiene la ejecución al encontrar el primer error.

Facilita la depuración y las pruebas rápidas.

Depende del intérprete instalado para ejecutarse.

Ejemplo: Python (CPython), donde el código se compila a bytecode y la máquina virtual lo interpreta.



COMPILADOR

Un compilador es un procesador de lenguaje que traduce todo el código fuente a un lenguaje objeto (código máquina o intermedio) antes de su ejecución, generando un programa ejecutable.

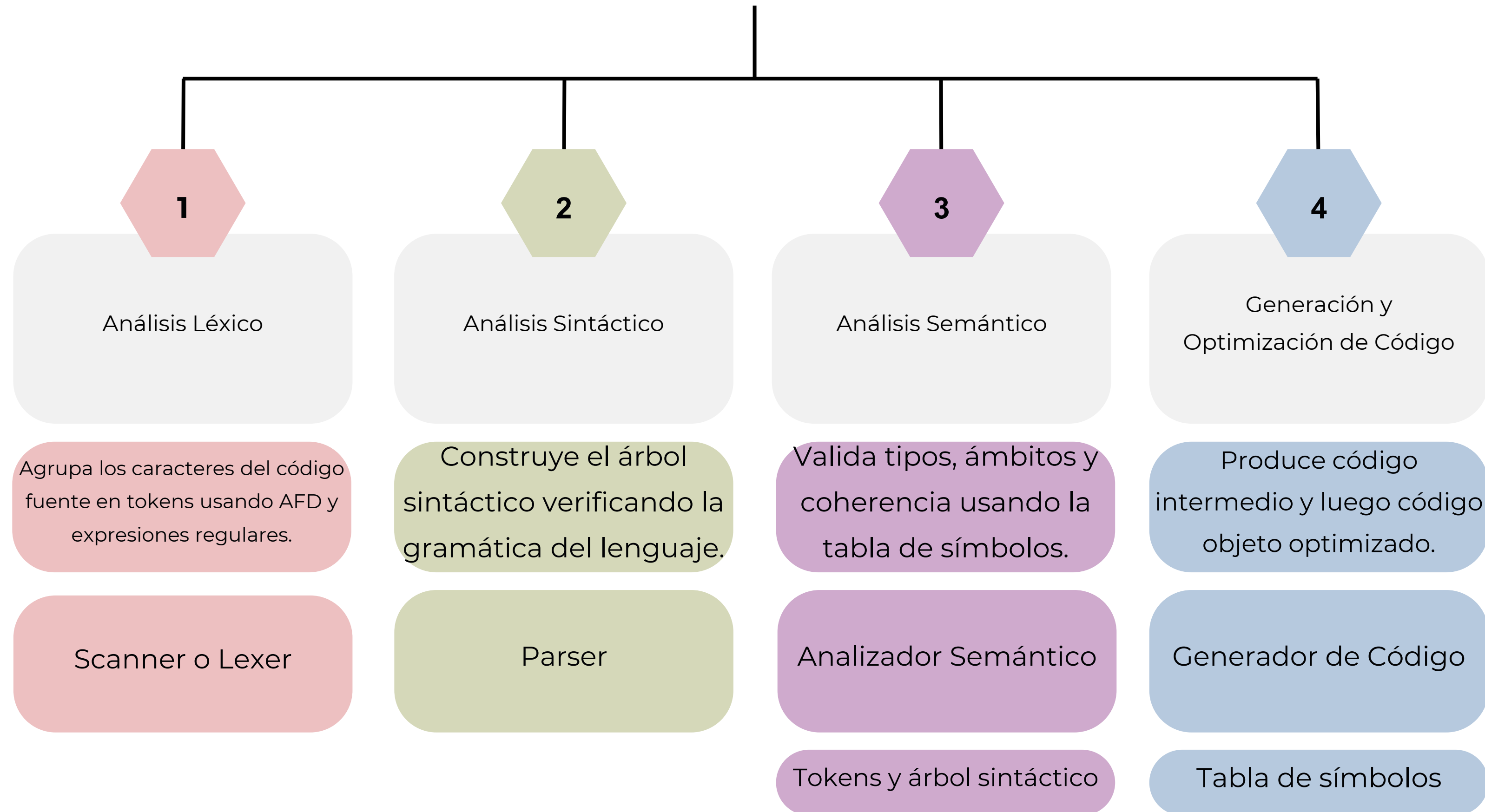
Analiza el programa completo y reporta todos los errores encontrados.

El ejecutable resultante se ejecuta de forma rápida e independiente.

Aplica optimizaciones sobre el código generado.

Ejemplos: C, C++, Rust y Go. En Python, la compilación a bytecode (.pyc) es una etapa previa a la interpretación.

FASES DEL COMPILADOR



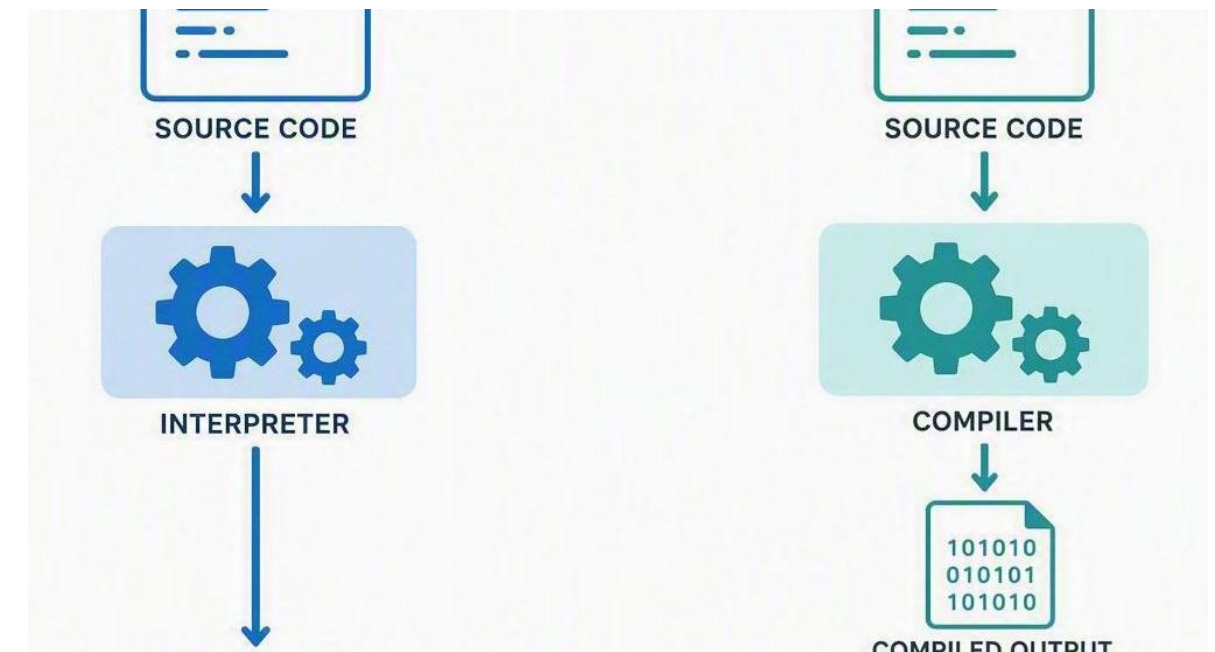
DIFERENCIAS

Traducción: el compilador procesa todo el programa antes de ejecutarlo; el intérprete traduce y ejecuta instrucción por instrucción.

Errores: el compilador los reporta todos al compilar; el intérprete se detiene en el primero.

Velocidad: el ejecutable compilado es más rápido; el intérprete es más flexible.

Depuración: el intérprete permite probar de inmediato; el compilador exige recompilar.



EJEMPLOS Y HERRAMIENTAS



EJEMPLOS

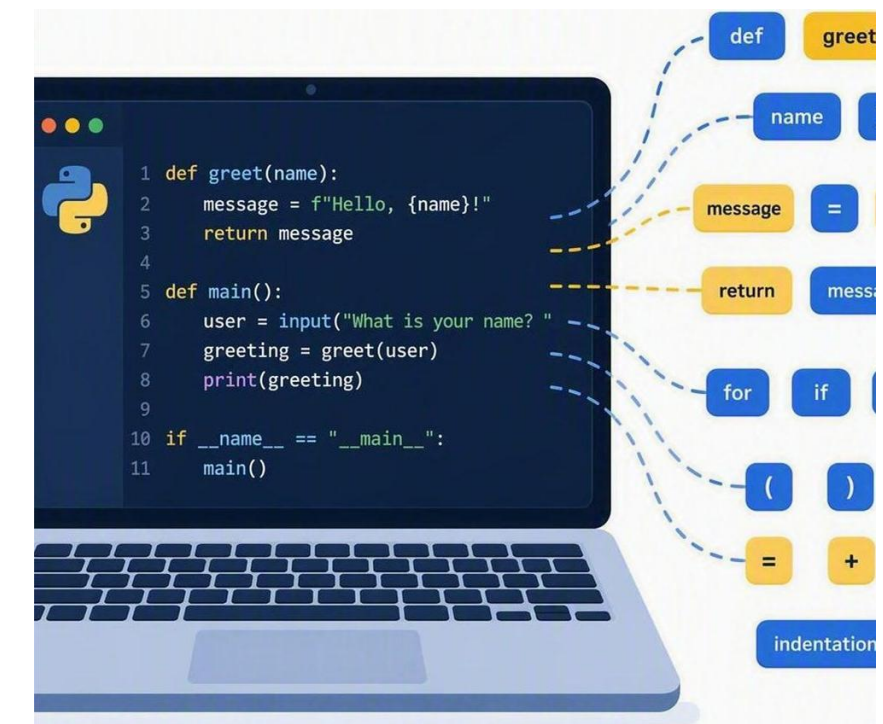
Lenguajes interpretados: Python, JavaScript, Ruby y PHP.

Lenguajes compilados: C, C++, Rust, Go y Pascal.

Modelo híbrido: Python compila a bytecode (.pyc) y la máquina virtual lo ejecuta; Java hace algo similar con la JVM.

Ejemplo en Python: `python programa.py` ejecuta el código con el intérprete, mientras que `py_compile.compile('programa.py')` genera el bytecode.

Ejemplo de fase léxica en Python: recorrer la cadena de entrada carácter por carácter y emitir tokens como NUMERO, IDENTIFICADOR y OPERADOR.



HERRAMIENTAS

PLY (Python Lex-Yacc): implementación de Lex y Yacc en Python para construir analizadores léxicos y sintácticos.

ANTLR: genera analizadores a partir de una gramática, con destino a Python, Java y otros lenguajes.

Lex y Yacc / Flex y Bison: herramientas clásicas para scanner y parser.

Módulos de Python: re para expresiones regulares y tokenize para análisis léxico.

Graphviz: útil para visualizar autómatas (AFD) y árboles sintácticos.



JERARQUÍA DE CHOMSKY

Clasificación propuesta por Noam Chomsky que ordena las gramáticas formales según su poder expresivo y la máquina que las reconoce.

Tipo 0 – Sin restricción: reconocidas por la Máquina de Turing; lenguajes recursivamente enumerables.

Tipo 1 – Sensible al contexto: reconocidas por una Máquina de Turing acotada.

Tipo 2 – Libre de contexto: reconocidas por un autómata de pila; base de los parsers.

Tipo 3 – Regular: reconocidas por AFD, AFND o expresiones regulares; base de los analizadores léxicos.



CLASIFICACIÓN DE GRAMÁTICAS



Una gramática se define como $G = (N, T, P, S)$: no terminales, terminales, producciones y símbolo inicial.

Gramática regular: producciones $A \rightarrow aB$ o $A \rightarrow a$; se usa en el análisis léxico.

Gramática libre de contexto: producciones $A \rightarrow \alpha$; describe la sintaxis del lenguaje.

Gramática sensible al contexto: la producción depende del entorno del símbolo.

Gramática sin restricción: sin limitaciones en sus producciones.



CONCEPTOS CLAVE APRENDIDOS

Intérprete

Traduce y ejecuta instrucción por instrucción.

Se detiene en el primer error.

Ejemplo: Python.

Compilador

Traduce todo el programa antes de ejecutarlo.

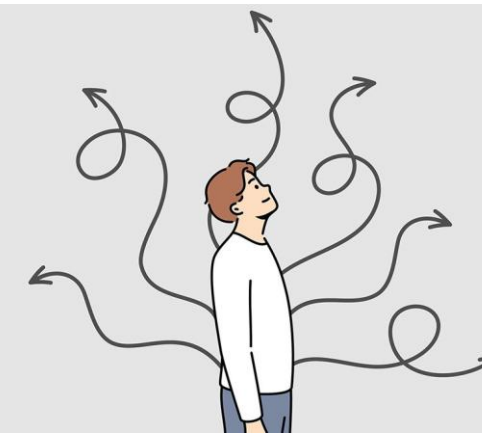
Genera un ejecutable optimizado.

Ejemplos: C, C++, Rust.

Estructura del compilador

Léxico, sintáctico, semántico y generación de código.

Tabla de símbolos y manejo de errores en todas las fases.



CONCEPTOS CLAVE APRENDIDOS

Herramientas

PLY y ANTLR para generar analizadores.

Módulos re y tokenize de Python.

Jerarquía de Chomsky

Tipo 0: sin restricción (Máquina de Turing).

Tipo 1: sensible al contexto.

Tipo 2: libre de contexto (parser).

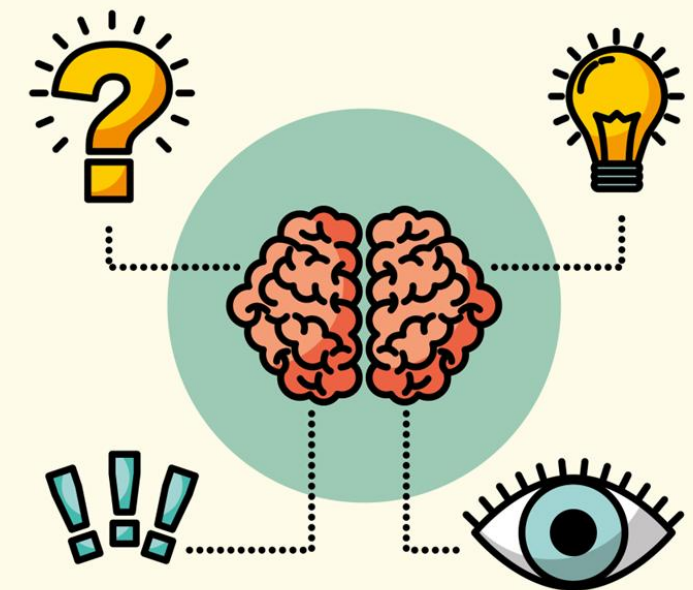
Tipo 3: regular (AFD y expresiones regulares).



VALORES

Paciencia: capacidad de mantener la calma, la serenidad y el control emocional ante situaciones difíciles, retrasos o frustraciones cuando los resultados tardan en llegar.

Depurar cada estado de un AFD exige probar, corregir y volver a intentar: la paciencia sostiene ese proceso.



REFERENCIAS

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008). Compiladores: principios, técnicas y herramientas (2a ed.). Pearson.
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). Introducción a la teoría de autómatas, lenguajes y computación (3a ed.). Pearson.





¡GRACIAS POR SU ATENCIÓN!

DUDAS

**RECUERDA QUE TENEMOS NUESTRO FORO SEMANAL DONDE PUEDES CONSULTAR CUALQUIER
DUDA QUE TE SURJA EN LA SEMANA**